

# AI DONATION TRACKER

## SYSTEM DOCUMENTATION

---

### Table of Contents

|                                        |    |
|----------------------------------------|----|
| 1. Introduction .....                  | 4  |
| 2. Purpose of the Application .....    | 5  |
| 3. Problem Statement .....             | 5  |
| 4. Proposed Solution .....             | 6  |
| 5. Main Features .....                 | 6  |
| 5.1 Donation Recording .....           | 6  |
| 5.2 Donation Summary .....             | 6  |
| 5.3 Recent Donations .....             | 7  |
| 5.4 AI Donation Assistant .....        | 7  |
| 5.5 Cardano Wallet Connection .....    | 7  |
| 5.6 ADA Donations .....                | 7  |
| 5.7 Transaction Verification .....     | 7  |
| 6. System Requirements .....           | 7  |
| 6.1 Hardware Requirements .....        | 8  |
| 6.2 Software Requirements .....        | 8  |
| 7. User Interface Design .....         | 8  |
| 8. HTML Structure .....                | 8  |
| 9. CSS Styling .....                   | 9  |
| 10. JavaScript Functionality .....     | 10 |
| 10.1 Backend Connection .....          | 11 |
| 10.2 Cardano Configuration .....       | 11 |
| 10.3 Status Messages .....             | 11 |
| 10.4 Sending Data to the Backend ..... | 12 |
| 10.5 Retrieving Data .....             | 12 |
| 10.6 HTML Security .....               | 12 |
| 11. Donation Management .....          | 12 |

|                                                |    |
|------------------------------------------------|----|
| 11.1 Loading Donations .....                   | 13 |
| 11.2 Displaying Donations .....                | 13 |
| 11.3 Cardano Verification Display .....        | 13 |
| 12. Saving a Donation .....                    | 13 |
| 13. AI Donation Assistant .....                | 14 |
| 14. Cardano Blockchain Integration .....       | 14 |
| 15. Cardano Wallet Integration .....           | 15 |
| 15.1 Connecting a Wallet .....                 | 15 |
| 15.2 Reading Wallet Balance .....              | 16 |
| 16. Cardano Donation Transaction Process ..... | 16 |
| 16.1 Protocol Parameters .....                 | 17 |
| 16.2 UTXO Selection .....                      | 17 |
| 16.3 Charity Output .....                      | 18 |
| 17. Transaction Metadata .....                 | 18 |
| 18. Change Address .....                       | 18 |
| 19. Unsigned Transaction .....                 | 19 |
| 20. Transaction Signing .....                  | 19 |
| 21. Transaction Submission .....               | 19 |
| 22. Cardano Transaction Verification .....     | 20 |
| 23. PHP Backend Implementation .....           | 20 |
| 24. Backend Configuration .....                | 21 |
| 24.1 Environment Variables .....               | 21 |
| 24.2 Security Headers .....                    | 22 |
| 24.3 CORS .....                                | 22 |
| 25. Retrieving Donations .....                 | 22 |
| 25.1 Donation Limit .....                      | 22 |
| 25.2 Total Donation Calculation .....          | 23 |
| 26. Saving Donations .....                     | 23 |
| 26.1 Donation Validation .....                 | 23 |
| 26.2 Prepared Statements .....                 | 24 |
| 27. Database Implementation .....              | 24 |
| 28. Donations Table .....                      | 24 |
| 29. AI Queries Table .....                     | 25 |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>30. Cardano Database Information .....</b>               | <b>25</b> |
| <b>31. Transaction Uniqueness .....</b>                     | <b>26</b> |
| <b>32. Cardano Verification Backend .....</b>               | <b>26</b> |
| <b>33. System Architecture .....</b>                        | <b>26</b> |
| <b>34. Frontend Layer .....</b>                             | <b>27</b> |
| <b>35. Backend Layer .....</b>                              | <b>27</b> |
| <b>36. Database Layer .....</b>                             | <b>28</b> |
| <b>37. Blockchain Layer .....</b>                           | <b>28</b> |
| <b>38. Complete Donation Workflow .....</b>                 | <b>29</b> |
| <b>39. Complete ADA Donation Workflow .....</b>             | <b>29</b> |
| <b>40. Complete Transaction Verification Workflow .....</b> | <b>31</b> |
| <b>41. Security Design .....</b>                            | <b>31</b> |
| <b>42. Cardano Security Flow .....</b>                      | <b>32</b> |
| <b>43. Error Handling .....</b>                             | <b>33</b> |
| <b>44. Technology Stack .....</b>                           | <b>34</b> |
| <b>45. Functional Requirements .....</b>                    | <b>34</b> |
| <b>46. Non-Functional Requirements .....</b>                | <b>35</b> |
| <b>47. Development Limitations .....</b>                    | <b>35</b> |
| <b>47.1 Preprod Network .....</b>                           | <b>35</b> |
| <b>47.2 Configuration Placeholders .....</b>                | <b>36</b> |
| <b>47.3 Simplified UTXO Selection .....</b>                 | <b>36</b> |
| <b>47.4 Wallet Selection .....</b>                          | <b>36</b> |
| <b>47.5 Blockfrost Browser Access .....</b>                 | <b>36</b> |
| <b>47.6 AI Endpoint .....</b>                               | <b>36</b> |
| <b>48. Future Improvements .....</b>                        | <b>36</b> |
| <b>48.1 User Accounts .....</b>                             | <b>36</b> |
| <b>48.2 Donation Receipts .....</b>                         | <b>37</b> |
| <b>48.3 Advanced AI Assistant .....</b>                     | <b>37</b> |
| <b>48.4 Admin Dashboard .....</b>                           | <b>37</b> |
| <b>48.5 Improved Blockchain Integration .....</b>           | <b>37</b> |
| <b>48.6 Mainnet Support .....</b>                           | <b>37</b> |
| <b>48.7 Better Wallet Selection .....</b>                   | <b>37</b> |
| <b>48.8 Improved Transaction Matching .....</b>             | <b>37</b> |

|                                             |           |
|---------------------------------------------|-----------|
| <b>49. Overall System Workflow .....</b>    | <b>38</b> |
| <b>50. Project Structure .....</b>          | <b>39</b> |
| <b>51. Component Responsibilities .....</b> | <b>40</b> |
| <b>52. Overall Evaluation .....</b>         | <b>41</b> |
| <b>53. Conclusion .....</b>                 | <b>41</b> |

## **1. Introduction**

The **AI Donation Tracker** is a web-based donation management application that combines traditional web technologies, artificial intelligence and Cardano blockchain technology.

The system allows users to record donations, view recent donations, interact with an AI Donation Assistant, connect a Cardano wallet, make ADA donations and verify Cardano transactions.

The application uses HTML and CSS for the user interface, JavaScript for client-side functionality, PHP for server-side processing, MySQL/MariaDB for data storage and Cardano blockchain technology for transparent cryptocurrency donation verification.

The main purpose of the application is to provide a simple and transparent way of managing donations while demonstrating how blockchain technology can be integrated into a modern web application.

---

## 2. Purpose of the Application

The purpose of the AI Donation Tracker is to provide a central platform where donations can be recorded and monitored.

The system is designed to:

- Record donor information.
  - Record donation amounts.
  - Record the purpose of donations.
  - Display recent donations.
  - Calculate the total amount of donations.
  - Provide an AI assistant for donation-related questions.
  - Allow users to connect a Cardano wallet.
  - Allow users to donate ADA.
  - Verify Cardano transactions.
  - Store verified blockchain transaction information.
  - Improve transparency in donation management.
- 

## 3. Problem Statement

Traditional donation systems can make it difficult for users to track donations and verify whether cryptocurrency payments have actually been received.

Some donation systems rely completely on manually maintained records. This can make verification difficult and can reduce transparency.

There is therefore a need for a system that combines normal donation record management with blockchain verification.

The AI Donation Tracker addresses this problem by allowing donation records to be stored in a database while Cardano transactions can be verified on the blockchain.

---

## 4. Proposed Solution

The proposed solution is a web application that combines four main technologies:

1. **Web frontend**
2. **PHP backend**
3. **Database**
4. **Cardano blockchain**

The frontend provides the user interface.

The PHP backend handles server-side operations and database communication.

The database stores donation and AI-related information.

Cardano provides blockchain-based transaction records and verification.

The AI assistant provides users with an additional way to obtain information about donations.

---

## 5. Main Features

The application provides the following main features:

### 5.1 Donation Recording

Users can enter:

- Donor name
- Donation amount
- Donation purpose

The information is validated and saved in the database.

### 5.2 Donation Summary

The application calculates and displays the total value of recorded donations in South African Rand.

## 5.3 Recent Donations

Users can view recently recorded donations in a table.

The table contains:

- Donor
- Amount
- Purpose
- Date
- Cardano verification status

## 5.4 AI Donation Assistant

Users can ask questions through the AI Donation Assistant.

The question is sent to the backend and an AI-generated answer can be returned to the user.

## 5.5 Cardano Wallet Connection

Users can connect a Cardano wallet such as:

- Nami
- Eternl
- Lace

The application uses the CIP-30 wallet interface to communicate with supported browser wallets.

## 5.6 ADA Donations

Connected users can enter an ADA amount and send a donation to the configured charity address.

## 5.7 Transaction Verification

Users can enter a Cardano transaction ID and request verification.

The backend checks the transaction through Blockfrost and determines whether the payment was confirmed.

---

# 6. System Requirements

## 6.1 Hardware Requirements

The application can run on a normal computer with:

- Processor capable of running a modern web browser
- At least 4 GB RAM
- Internet connection
- Keyboard and mouse

## 6.2 Software Requirements

The system requires:

- Modern web browser
  - Web server supporting PHP
  - PHP
  - MySQL or MariaDB
  - Cardano wallet extension for ADA donations
  - Blockfrost API access
  - Cardano Serialization Library
- 

## 7. User Interface Design

The application uses a dark-themed interface.

The main page is divided into several sections:

1. Donation Summary
2. Add a Donation
3. Recent Donations
4. AI Donation Assistant
5. Cardano

The design is intended to be simple and easy to navigate.

---

## 8. HTML Structure

The HTML file defines the main structure of the application.



The page includes the following sections:

### **Donation Summary**

Displays the total donation amount.

### **Add a Donation**

Contains:

- Donor name input
- Donation amount input
- Purpose dropdown
- Save Donation button

### **Recent Donations**

Displays donation records in a table.

### **AI Donation Assistant**

Contains:

- Question text area
- Ask AI button
- AI response area

### **Cardano**

Contains:

- Connect Wallet button
- Wallet status
- ADA amount input
- Donate with ADA button
- Transaction ID input
- Verify Transaction button

The HTML also loads the required JavaScript files and the Cardano Serialization Library.

---

## **9. CSS Styling**

The application uses a custom CSS stylesheet.

The main visual characteristics are:

- Dark navy background
- Dark blue panels
- White text
- Yellow-green headings
- Blue buttons
- Rounded corners
- Shadows
- Responsive layout

The page has a maximum width of approximately 720 pixels, keeping the content readable.

The application also includes responsive styling for smaller screens.

Tables can scroll horizontally on small devices to prevent the page layout from breaking.

---

## 10. JavaScript Functionality

JavaScript controls most of the client-side functionality.

The main JavaScript file is `app.js`.

It is responsible for:

- Sending requests to the backend.
  - Retrieving donation records.
  - Displaying donations.
  - Saving donations.
  - Communicating with the AI endpoint.
  - Connecting Cardano wallets.
  - Sending ADA donations.
  - Requesting transaction verification.
  - Displaying success and error messages.
-

## 10.1 Backend Connection

The JavaScript defines:

```
BACKEND_BASE_URL = 'backend'
```

This tells the application where the PHP backend endpoints are located.

The postJson() function sends POST requests.

The getJson() function sends GET requests.

This creates a simple communication layer between the frontend and backend.

---

## 10.2 Cardano Configuration

The Cardano configuration contains:

- Network
- Blockfrost project ID
- Charity address
- Backend URL

The project currently uses the **Cardano Preprod test network**.

This is appropriate for development and testing rather than real donations.

The configuration contains placeholders that must be replaced before the application can operate correctly.

---

## 10.3 Status Messages

The setStatus() function is used to display feedback to users.

Examples include:

- Donation saved.
- Thinking...
- Wallet connected.
- Transaction verified.
- Error messages.

Success and error messages use different CSS classes so that users can easily understand the result.

---

## 10.4 Sending Data to the Backend

JavaScript sends data to PHP using the `fetch()` API.

For example, when a donation is submitted, the frontend sends:

donorName

amount

purpose

to the donation-saving endpoint.

The backend validates and stores the information.

---

## 10.5 Retrieving Data

The application retrieves donation records from the PHP backend.

The returned JSON contains:

- Total donation amount
- Donation records
- Cardano transaction hash
- Verification status
- Donation date

The JavaScript then updates the page.

---

## 10.6 HTML Security

The `escapeHtml()` function is used before displaying donor information in the donation table.

This helps reduce the risk of malicious HTML or JavaScript being inserted into the page through user-controlled data.

This is an important client-side security measure.

---

# 11. Donation Management

The donation management feature allows users to create and view donation records.

When the page loads, JavaScript requests the latest donations from the backend.

The records are displayed in the Recent Donations table.

---

## 11.1 Loading Donations

The `loadDonations()` function calls the donation retrieval endpoint.

The backend queries the database and returns the donation information as JSON.

The frontend then calls `renderDonations()` to display the results.

---

## 11.2 Displaying Donations

Each donation displays:

- Donor name
- Amount
- Purpose
- Date
- Cardano status

The amount is formatted as South African Rand.

For example:

R500.00

---

## 11.3 Cardano Verification Display

If a donation has a verified Cardano transaction, the table displays a shortened transaction hash.

For example:

✓ 4f8a2c91ab...

If no verified Cardano transaction exists, the system displays:

—

---

# 12. Saving a Donation

When the user clicks **Save Donation**, JavaScript collects the form information.

The information is sent to:

save\_donation.php

The backend validates the information before inserting it into the database.

After successful insertion:

1. A success message is displayed.
  2. The form is cleared.
  3. Donations are reloaded.
  4. The new donation becomes visible in the table.
- 

## 13. AI Donation Assistant

The AI Donation Assistant allows users to ask donation-related questions.

The user enters a question into the text area and clicks **Ask AI**.

JavaScript sends the question to the AI backend endpoint.

The request contains:

question

The backend processes the request and returns an answer.

The answer is then displayed in the aiAnswer section.

The system also displays a temporary:

Thinking...

message while waiting for the response.

---

## 14. Cardano Blockchain Integration

Cardano is used to provide transparent cryptocurrency donation functionality.

The application uses:

- Cardano Preprod network
- CIP-30 wallet API
- Cardano Serialization Library
- Blockfrost API

The Cardano functionality is implemented in `cardano/offchain.js`.

The module is exposed through:

`window.CardanoOffChain`

This allows the normal JavaScript application to use the Cardano functions.

---

## 15. Cardano Wallet Integration

The application uses the Cardano CIP-30 wallet interface.

CIP-30 allows browser-based Cardano applications to communicate with wallet extensions.

The application checks:

`window.cardano`

to determine which wallet extensions are available.

The system can detect wallets that provide an `enable()` function.

The user can then connect to a supported wallet.

---

### 15.1 Connecting a Wallet

The wallet connection process is:

User clicks Connect Wallet



Application checks installed wallets



Wallet is selected



Wallet permission is requested



User approves connection



CIP-30 API is returned



Wallet is connected

If no wallet is installed, the application displays an error message.

---

## 15.2 Reading Wallet Balance

The application can request the connected wallet's balance.

Cardano values are converted using the Cardano Serialization Library.

The balance is returned in lovelace.

Cardano uses:

1 ADA = 1,000,000 lovelace

---

## 16. Cardano Donation Transaction Process

When a user makes an ADA donation, the application performs several steps.

Connect wallet



Enter ADA amount



Retrieve protocol parameters



Retrieve wallet UTXOs



Select UTXOs



Create charity output



Add transaction metadata



Add change address





Build unsigned transaction



Ask wallet to sign



Submit transaction



Verify transaction



Return transaction hash

---

## 16.1 Protocol Parameters

Before building a transaction, the application retrieves the latest Cardano protocol parameters through Blockfrost.

These parameters are required to correctly construct the transaction.

They include information related to:

- Transaction fees
  - Deposits
  - UTXO requirements
  - Maximum transaction size
  - Maximum value size
- 

## 16.2 UTXO Selection

Cardano uses the UTXO model.

The application retrieves spendable UTXOs from the user's wallet.

It selects enough UTXOs to cover:

Donation amount + estimated transaction fee

The current implementation uses an additional 500,000 lovelace buffer as a rough fee allowance.

If the wallet does not contain enough ADA, the transaction is rejected.

---

## 16.3 Charity Output

The transaction contains an output directed to the configured charity address.

The ADA donation amount is converted from ADA to lovelace before being added to the transaction.

The charity address is configured separately rather than being hardcoded throughout the code.

---

## 17. Transaction Metadata

The application attaches donor information and donation purpose to the Cardano transaction.

The metadata uses label:

674

The metadata contains:

donor

purpose

For example:

donor: "Example Donor"

purpose: "Education"

The application limits these values to 64 characters.

This allows donation information to be associated with the blockchain transaction.

---

## 18. Change Address

After the donation output is added, the remaining ADA must be returned to the donor.

The application requests the wallet's change address.

The change address is then used when building the transaction.

This prevents the remaining funds from being unintentionally sent to the charity.

---

## 19. Unsigned Transaction

The application builds the transaction before requesting the user's approval.

At this point the transaction is unsigned.

The unsigned transaction contains:

- Selected inputs
- Charity output
- Transaction metadata
- Change address

The transaction is then passed to the user's wallet for signing.

---

## 20. Transaction Signing

The transaction is signed using the connected Cardano wallet.

The most important security principle is that the donor's private key does **not** leave the wallet.

The process is:

Application creates transaction

↓

Wallet receives unsigned transaction

↓

User reviews and approves

↓

Wallet signs transaction

↓

Signed transaction returned

The private key remains inside the wallet extension.

---

## 21. Transaction Submission

After signing, the application attempts to submit the transaction through the wallet.

If wallet submission fails, the application can attempt to submit the signed transaction through Blockfrost.

After successful submission, a transaction hash is returned.

The transaction hash identifies the transaction on the Cardano blockchain.

---

## 22. Cardano Transaction Verification

The application provides a transaction verification feature.

The user enters a transaction ID.

The backend verifies the transaction through Blockfrost.

The verification process checks whether a payment was made to the configured charity address.

The result can include:

- Verification status
  - Block height
  - Amount paid in lovelace
  - Reason if verification fails
- 

## 23. PHP Backend Implementation

PHP provides the server-side functionality of the application.

The backend handles:

- Database communication
- Donation validation
- Donation storage
- Donation retrieval
- Cardano transaction verification
- JSON responses
- Security configuration

The main backend files are:

config.php

get\_donations.php

save\_donation.php

verify\_transaction.php

The project also uses supporting classes such as:

lib/Database.php

lib/BlockfrostClient.php

---

## 24. Backend Configuration

The config.php file is loaded by the PHP endpoints.

It performs several important functions.

These include:

- Loading environment variables
  - Setting security headers
  - Configuring CORS
  - Handling JSON requests
  - Returning JSON responses
  - Handling errors
- 

### 24.1 Environment Variables

Sensitive configuration is stored in a .env file.

The application can use environment variables for information such as:

ALLOWED\_ORIGINS

CHARITY\_RECEIVE\_ADDRESS

Database and API credentials can also be stored through environment configuration.

This is safer than hardcoding secrets directly into PHP source code.

---

## 24.2 Security Headers

The backend uses several HTTP security headers.

These include:

X-Content-Type-Options: nosniff

X-Frame-Options: DENY

Referrer-Policy: no-referrer-when-downgrade

These provide additional protection against common browser-based attacks.

---

## 24.3 CORS

The application uses an allowed-origin list.

Only origins specified in the configuration are allowed to make cross-origin requests.

This helps prevent unauthorised websites from accessing the backend API.

---

# 25. Retrieving Donations

The donation retrieval endpoint uses a GET request.

The backend queries the database and retrieves the latest donation records.

The query returns:

- ID
- Donor name
- Amount
- Purpose
- Cardano transaction hash
- Verification status
- Creation date

The records are sorted from newest to oldest.

---

## 25.1 Donation Limit

The backend supports a configurable number of returned donations.

The default is:

20 donations

The maximum allowed value is:

200 donations

This prevents unnecessarily large database queries.

---

## 25.2 Total Donation Calculation

The backend calculates the total donation amount using SQL.

The total is returned together with the donation records.

If there are no donations, the system returns zero rather than a null value.

The frontend displays the result in the Donation Summary section.

---

## 26. Saving Donations

The `save_donation.php` endpoint handles new donation records.

The endpoint only accepts POST requests.

It receives:

donorName

amount

purpose

The information is validated before being saved.

---

### 26.1 Donation Validation

The backend validates the donor name.

The name:

- Must not be empty.
- Must not exceed 120 characters.

The amount:

- Must be numeric.

- Must be greater than zero.

The purpose must be one of:

Food

Education

Healthcare

Shelter

Other

This prevents invalid information from being stored.

---

## 26.2 Prepared Statements

The backend uses PDO prepared statements when inserting data into the database.

Prepared statements help protect against SQL injection attacks.

The donation data is therefore passed as parameters rather than directly inserted into an SQL query.

---

## 27. Database Implementation

The application uses **MySQL/MariaDB**.

The database structure is defined in:

schema.sql

The database contains two main tables:

1. donations
  2. ai\_queries
- 

## 28. Donations Table

The donations table contains the following fields:

| Field | Data Type    | Description                |
|-------|--------------|----------------------------|
| id    | INT UNSIGNED | Unique donation identifier |



| Field           | Data Type       | Description                |
|-----------------|-----------------|----------------------------|
| donor_name      | VARCHAR(120)    | Name of donor              |
| amount          | DECIMAL(12,2)   | Donation amount in Rand    |
| purpose         | VARCHAR(60)     | Purpose of donation        |
| cardano_tx_hash | CHAR(64)        | Cardano transaction hash   |
| ada_lovelace    | BIGINT UNSIGNED | Cardano amount in lovelace |
| verified        | TINYINT(1)      | Verification status        |
| created_at      | TIMESTAMP       | Date and time created      |

The id field is the primary key.

---

## 29. AI Queries Table

The ai\_queries table stores AI assistant interactions.

It contains:

| Field      | Data Type    | Description         |
|------------|--------------|---------------------|
| id         | INT UNSIGNED | Unique query ID     |
| question   | TEXT         | User's question     |
| answer     | TEXT         | AI-generated answer |
| created_at | TIMESTAMP    | Date and time       |

This provides a database structure for recording AI interactions.

---

## 30. Cardano Database Information

The database stores Cardano information separately from the normal Rand donation amount.

The amount field represents the donation in South African Rand.

The ada\_lovelace field represents the amount received through Cardano.

The cardano\_tx\_hash stores the blockchain transaction identifier.

The verified field indicates whether the blockchain transaction has been confirmed.

---

## 31. Transaction Uniqueness

The `cardano_tx_hash` field has a unique constraint.

This helps prevent the same transaction from being associated with multiple donation records.

It provides an additional database-level protection against duplicate transaction records.

---

## 32. Cardano Verification Backend

The transaction verification endpoint accepts a transaction ID.

The transaction ID is validated using a regular expression.

The expected format is:

64 hexadecimal characters

The backend then retrieves the configured charity address.

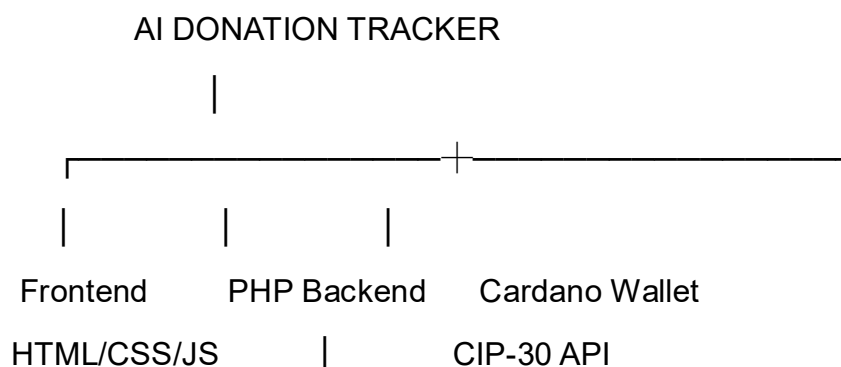
Blockfrost is used to verify whether the transaction contains a payment to the charity address.

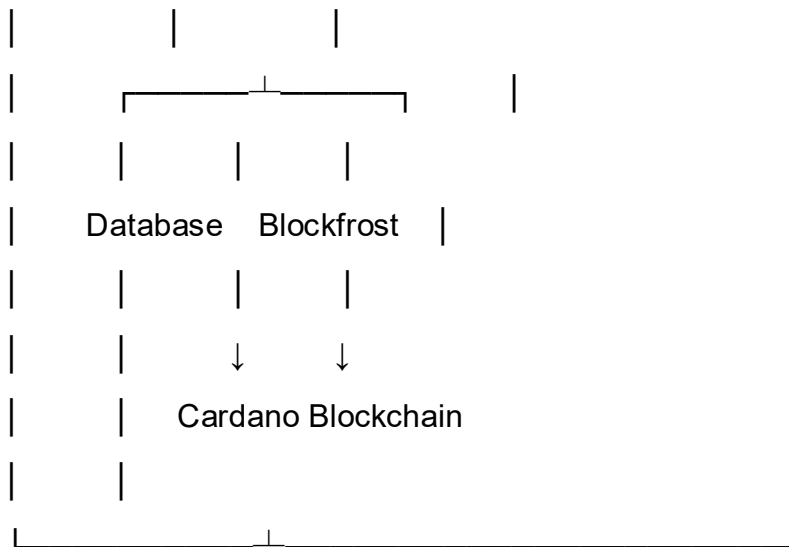
If confirmed, the database can be updated with the transaction hash, lovelace amount and verification status.

---

## 33. System Architecture

The overall system consists of four major layers.





Each component has a specific responsibility.

## 34. Frontend Layer

The frontend is responsible for:

- User interaction
- Form input
- Displaying donation information
- Displaying AI responses
- Connecting to Cardano wallets
- Requesting ADA transactions
- Displaying transaction verification results

The frontend uses:

HTML

CSS

JavaScript

## 35. Backend Layer

The PHP backend is responsible for:

- Request processing
  - Input validation
  - Database operations
  - Transaction verification
  - JSON responses
  - Security configuration
- 

## 36. Database Layer

The database stores:

- Donation information
  - AI queries
  - Cardano transaction hashes
  - ADA amounts
  - Verification status
  - Dates
- 

## 37. Blockchain Layer

Cardano provides:

- ADA transactions
- Transaction hashes
- On-chain transaction records
- Blockchain-based verification
- Transaction metadata

Blockfrost acts as the communication layer between the application and the Cardano blockchain.

---

## 38. Complete Donation Workflow

The normal donation workflow is:

User opens application



User enters donor information



User enters donation amount



User selects purpose



Clicks Save Donation



JavaScript sends POST request



PHP validates information



Database stores donation



PHP returns JSON response



JavaScript displays success



Donation list is refreshed

---

## 39. Complete ADA Donation Workflow

The Cardano donation workflow is:

User connects Cardano wallet



Wallet permission granted



User enters ADA amount



Application retrieves protocol parameters



Application retrieves wallet UTXOs



UTXOs are selected



Charity output is created



Donation metadata is added



Change address is added



Unsigned transaction is built



Wallet signs transaction



User approves transaction



Signed transaction submitted



Transaction hash returned



Backend verifies transaction



Database updated



Donation marked as verified

---

## 40. Complete Transaction Verification Workflow

The verification workflow is:

User enters transaction ID



JavaScript sends request



PHP receives transaction ID



Transaction ID is validated



Charity address is loaded



Blockfrost checks transaction



Payment is checked



Verification result returned



Database updated if appropriate



Frontend displays result

---

## 41. Security Design

Security is an important part of the application.

The system uses several security mechanisms.

### **Input Validation**

The backend checks user input before storing it.

### **Prepared Statements**

PDO prepared statements reduce the risk of SQL injection.

### **HTML Escaping**

User-controlled information is escaped before being inserted into the page.

### **Environment Variables**

Configuration and sensitive information can be stored outside the source code.

### **CORS Restrictions**

Only configured origins are allowed to access the backend cross-origin.

### **Security Headers**

HTTP security headers provide additional browser protection.

### **Wallet Security**

Private Cardano wallet keys remain inside the user's wallet extension.

---

## **42. Cardano Security Flow**

The Cardano security model is particularly important.

Application



Creates unsigned transaction



User's Cardano Wallet



User reviews transaction



User approves



Wallet signs transaction





Signed transaction returned



Blockchain

The application never needs the user's private key.

This is one of the major security advantages of using a browser wallet through CIP-30.

---

## 43. Error Handling

The application includes error handling on both the frontend and backend.

Examples include:

### **Missing Donation Information**

Donor name is required.

### **Invalid Amount**

Amount must be a positive number.

### **Invalid Purpose**

Purpose must be one of the allowed options.

### **No Wallet**

No Cardano wallet extension found.

### **Insufficient ADA**

Not enough ADA in wallet to cover this donation plus fees.

### **Invalid Transaction ID**

That doesn't look like a valid Cardano transaction ID.

### **Backend Errors**

The PHP backend returns HTTP status codes and JSON error messages.

This allows JavaScript to display meaningful feedback to the user.

---

## 44. Technology Stack

| Technology                    | Purpose                      |
|-------------------------------|------------------------------|
| HTML5                         | Structure of the application |
| CSS3                          | Interface styling            |
| JavaScript                    | Client-side functionality    |
| PHP                           | Server-side processing       |
| MySQL/MariaDB                 | Database                     |
| Cardano                       | Blockchain                   |
| CIP-30                        | Wallet communication         |
| Blockfrost                    | Cardano API                  |
| Cardano Serialization Library | Transaction construction     |
| AI API/backend                | AI assistant functionality   |

---

## 45. Functional Requirements

The system should be able to:

1. Allow users to enter donation information.
2. Validate donation information.
3. Save donations to the database.
4. Display recent donations.
5. Calculate total donations.
6. Allow users to ask questions through the AI assistant.
7. Detect supported Cardano wallets.
8. Connect to a Cardano wallet.
9. Read wallet information.
10. Create ADA donation transactions.
11. Allow users to approve transactions through their wallets.
12. Submit Cardano transactions.

13. Verify Cardano transaction IDs.
  14. Store verified transaction information.
  15. Display Cardano verification status.
- 

## **46. Non-Functional Requirements**

### **Security**

The application should protect user and system information.

### **Usability**

The interface should be simple and easy to understand.

### **Reliability**

The system should provide appropriate error messages when operations fail.

### **Performance**

Database queries should avoid retrieving unnecessarily large amounts of information.

### **Scalability**

The system should be capable of being expanded with additional donation features.

### **Maintainability**

The frontend, backend, database and blockchain components are separated to make the system easier to maintain.

---

## **47. Development Limitations**

Although the application demonstrates the main concept successfully, there are several limitations in the current implementation.

### **47.1 Preprod Network**

The application is configured for the Cardano Preprod network.

This means it is designed primarily for testing.

It should not be considered a production system for real donations until the network configuration is changed and thoroughly tested.

## 47.2 Configuration Placeholders

The current code contains placeholders for:

Blockfrost Project ID

Charity Address

These must be replaced with valid configuration values.

## 47.3 Simplified UTXO Selection

The application performs basic UTXO selection.

A production application would require more advanced handling of:

- Fees
- UTXOs
- Multi-asset transactions
- Protocol parameter changes
- Transaction size

## 47.4 Wallet Selection

The current implementation selects the first available wallet for simplicity.

A production version should provide a wallet selection interface.

## 47.5 Blockfrost Browser Access

The current frontend uses a browser-safe/read-only Blockfrost project ID.

For a production system, blockchain API access should be carefully controlled and sensitive credentials should not be exposed unnecessarily.

## 47.6 AI Endpoint

The AI functionality depends on the backend AI implementation.

The actual `ask_ai.php` implementation should be connected to an appropriate AI service before the feature is considered complete.

---

# 48. Future Improvements

Several improvements can be made to the system.

## 48.1 User Accounts

Users could create accounts and securely log in.

This would allow users to view their own donation history.

## **48.2 Donation Receipts**

The system could generate downloadable donation receipts.

## **48.3 Advanced AI Assistant**

The AI assistant could provide:

- Donation recommendations
- Donation summaries
- Charity information
- Spending explanations
- Frequently asked questions

## **48.4 Admin Dashboard**

An administrator dashboard could allow authorised users to:

- View all donations
- Verify transactions
- Manage donation categories
- View statistics
- Monitor blockchain transactions

## **48.5 Improved Blockchain Integration**

A production version could use a higher-level Cardano library such as Lucid or Mesh.

This could simplify transaction construction and provide more robust handling of Cardano protocol requirements.

## **48.6 Mainnet Support**

After extensive testing, the application could be configured for Cardano Mainnet.

This should only be done after proper security testing and deployment preparation.

## **48.7 Better Wallet Selection**

A wallet selection screen could allow users to choose between available wallets instead of automatically selecting the first detected wallet.

## **48.8 Improved Transaction Matching**

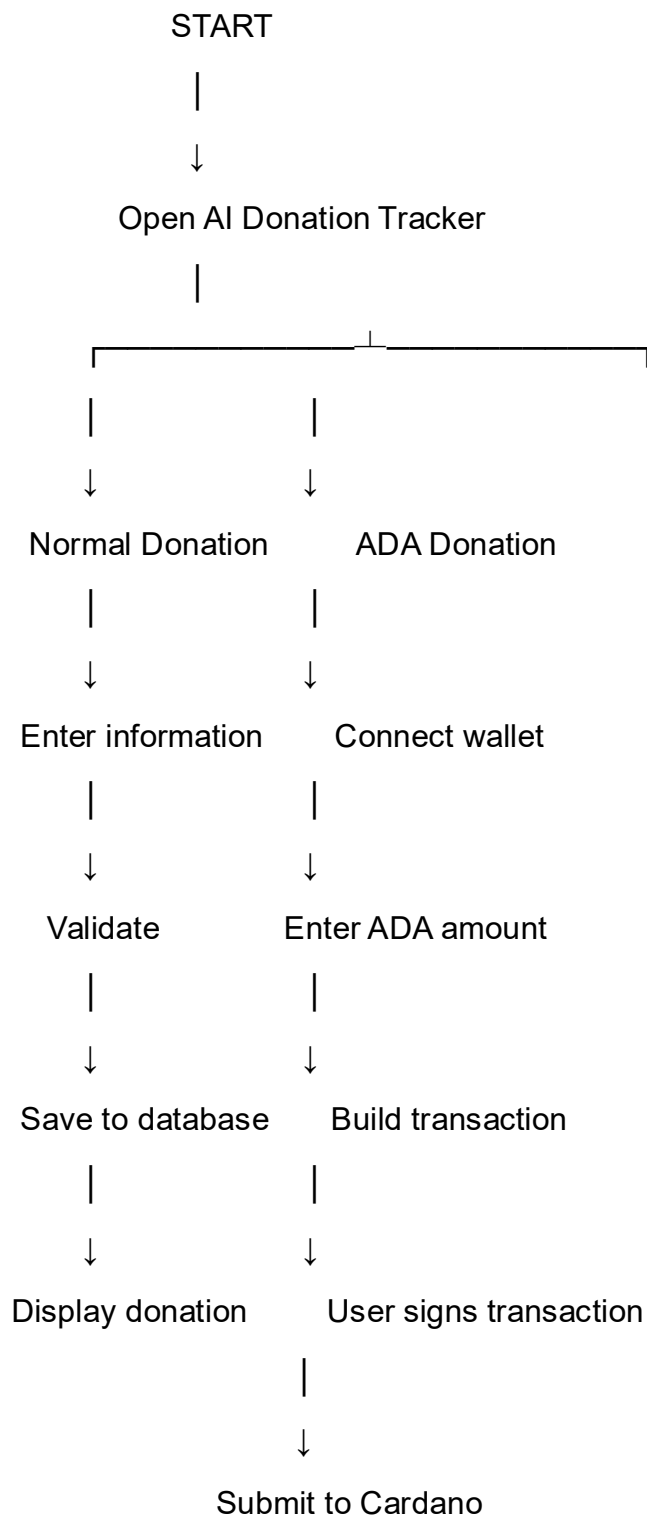
The system could associate a blockchain transaction with a specific donation ID during the donation process.

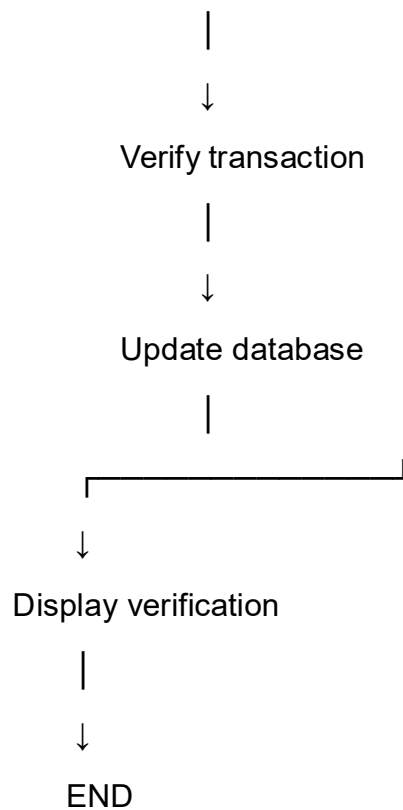
This would make the database relationship between the Rand donation record and Cardano transaction clearer.

---

## 49. Overall System Workflow

The complete application can be summarised as follows:





---

## 50. Project Structure

A simplified project structure is:

AI Donation Tracker/

```
|
|
|—— index.html
|—— style.css
|—— app.js
|
|—— cardano/
|   |—— offchain.js
|
|—— backend/
|   |—— config.php
|   |—— get_donations.php
```

```
|   ├── save_donation.php
|   ├── verify_transaction.php
|   |
|   └── lib/
|       ├── Database.php
|       └── BlockfrostClient.php
|
└── schema.sql
```

The project structure separates the different parts of the application.

---

## 51. Component Responsibilities

| Component              | Responsibility                            |
|------------------------|-------------------------------------------|
| index.html             | User interface structure                  |
| style.css              | User interface design                     |
| app.js                 | Frontend application logic                |
| cardano/offchain.js    | Cardano wallet and transaction operations |
| config.php             | Backend configuration and security        |
| get_donations.php      | Retrieves donations                       |
| save_donation.php      | Saves donations                           |
| verify_transaction.php | Verifies Cardano transactions             |
| Database.php           | Database connection                       |
| BlockfrostClient.php   | Communication with Blockfrost             |
| schema.sql             | Database structure                        |

---



## 52. Overall Evaluation

The AI Donation Tracker successfully demonstrates the integration of several modern technologies into one application.

The system combines normal database-based donation management with blockchain-based verification.

The database provides structured and persistent storage, while Cardano provides a transparent transaction record.

The AI assistant adds an intelligent interaction component that can help users obtain information.

The separation between frontend, backend, database and blockchain components also provides a foundation for future development.

---

## 53. Conclusion

The AI Donation Tracker is a web-based donation management system that demonstrates how artificial intelligence, traditional web development and blockchain technology can work together.

The application allows users to record donations and view donation information through a simple web interface. The PHP backend validates and processes information, while MySQL/MariaDB provides persistent storage.

Cardano provides an additional layer of transparency by allowing ADA donations to be recorded on the blockchain. Users can connect their Cardano wallets, approve transactions and send ADA directly to the configured charity address.

The system also verifies Cardano transaction IDs through Blockfrost and can store verified blockchain information in the database.

Security has been considered throughout the system. User input is validated, prepared SQL statements are used, security headers are configured, and private Cardano wallet keys remain inside the user's wallet.

Although the current implementation is primarily a development and testing system, it provides a strong foundation for a more advanced production donation platform. Future improvements could include user accounts, administration features, donation receipts, enhanced AI functionality, improved blockchain transaction handling and eventual Cardano Mainnet deployment.

Overall, the project demonstrates how a donation platform can use **AI for intelligent assistance, databases for record management, and Cardano blockchain technology for transparent and verifiable cryptocurrency donations.**

